

Analyzing Sequential vs. Independent Sentence Classification with End-to-End MLOps: An Empirical Study Using the PubMed RCT Dataset

Muhammad Abubakar, Ayaan Khan, Sahil Kumar

Department of Computer Science

FAST National University of Computer and Emerging Sciences

Islamabad, Pakistan

i222003@nu.edu.pk, i222066@nu.edu.pk, i222048@nu.edu.pk

Abstract—Machine learning models deployed in production environments require robust operational practices to ensure reliability, scalability, and maintainability. This paper presents an end-to-end MLOps pipeline for sequential document classification, inspired by the Intelligent Tendering system developed by Fraunhofer IAIS and Schneider Electric. We implement and compare two BERT-based classification architectures—a Single Sentence Classifier (SinSent) and a Sequential Sentence Classifier (SeqSent)—on the PubMed RCT dataset for biomedical abstract sentence role classification. Our pipeline integrates five core MLOps tools: MLflow for experiment tracking, Docker for containerization, Prometheus for metrics collection, Grafana for monitoring dashboards, and GitHub Actions for CI/CD automation. Experimental results demonstrate that SinSent achieves a micro-F1 of 0.455 compared to SeqSent’s 0.383 under resource-constrained conditions, while SeqSent exhibits superior recall on the RESULTS class (0.78 vs. 0.69). We analyze the impact of dataset size, model capacity, and architectural complexity on the relative performance of context-aware versus independent classification, and discuss implications for MLOps-driven model selection in production environments.

Index Terms—MLOps, Natural Language Processing, Sequential Sentence Classification, BERT, Experiment Tracking, CI/CD, Container Orchestration, Model Monitoring

I. INTRODUCTION

The deployment of machine learning (ML) models in production environments remains a significant challenge for organizations seeking to derive value from their data science efforts. While considerable research attention has been devoted to improving model architectures and training techniques, the operational aspects of deploying, monitoring, and maintaining ML systems are equally critical for long-term success [2]. Machine Learning Operations (MLOps) has emerged as a discipline that addresses these challenges through standardized practices for the full ML lifecycle [3].

In this work, we implement and analyze the ML system architecture presented by Martens et al. [1], who developed a production-ready classification system for Request for Quote (RFQ) documents in electrical engineering. Their system compares two classification approaches: a Single Sentence Classifier (SinSent) that classifies each document position independently, and a Sequential Sentence Classifier (SeqSent)

that leverages contextual information from surrounding positions. The original work demonstrated that the context-aware SeqSent model outperforms SinSent (F1: 0.49 vs. 0.41) when deployed with a comprehensive MLOps infrastructure.

Our contributions are threefold: (1) We replicate the dual-model architecture on the publicly available PubMed RCT dataset [4], enabling reproducible analysis of sequential vs. independent sentence classification. (2) We implement a complete MLOps pipeline integrating MLflow, Docker, Prometheus, Grafana, and GitHub Actions, demonstrating practical MLOps competency. (3) We provide empirical analysis of how resource constraints (limited data, lightweight models) affect the relative advantage of context-aware classification.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes our methodology including model architectures and MLOps pipeline design. Section IV presents experimental results and analysis. Section V discusses the MLOps infrastructure in detail. Section VI provides discussion and lessons learned. Section VII concludes the paper.

II. RELATED WORK

A. Sequential Sentence Classification

Sequential sentence classification involves assigning labels to sentences within a document while considering their contextual relationships. Cohan et al. [5] introduced pretrained language models for this task, demonstrating that sequential context significantly improves classification accuracy over independent sentence classification. Their work on the PubMed RCT dataset showed that BiLSTM layers on top of sentence embeddings effectively capture document-level dependencies. Dernoncourt and Lee [4] established the PubMed RCT dataset as a benchmark for sequential sentence classification in biomedical abstracts, categorizing sentences into five rhetorical roles.

B. MLOps Practices

Kreuzberger et al. [2] provide a comprehensive overview of MLOps, defining it as a set of practices that combines

ML, DevOps, and data engineering to deploy and maintain ML systems reliably. Symeonidis et al. [3] survey MLOps tools and frameworks, categorizing them by functionality. Testi et al. [7] examine MLOps maturity models and identify key practices for production ML systems. Alla and Adari [8] discuss practical implementation of MLOps pipelines with emphasis on experiment tracking and model versioning.

C. Language Models in Production

Devlin et al. [6] introduced BERT, the foundation for our classification models. Paß and Giesselbach [9] discuss foundation models for NLP including deployment considerations. Bommasani et al. [10] analyze opportunities and risks of foundation models, including operational challenges. Yoon et al. [11] present a production NER system using distilled BERT models with enterprise MLOps practices, demonstrating the practical intersection of NLP and MLOps.

D. The Intelligent Tendering System

Martens et al. [1] present the most directly relevant work—a production-ready classification system for RFQ documents using GBERT with MLOps practices including GitLab CI, MLflow, Docker, and Kubernetes. Their sequential model achieved an F1 of 0.49 versus 0.41 for the single sentence model on proprietary electrical engineering data. Our work extends their approach by replicating the architecture on a public dataset and analyzing performance under different resource conditions.

III. METHODOLOGY

A. Research Questions

We formulate the following research questions:

- **RQ1:** Does context-aware sequential classification (SeqSent) consistently outperform independent classification (SinSent) across different domains and resource conditions?
- **RQ2:** What is the impact of model capacity and training data volume on the relative advantage of sequential classification?
- **RQ3:** How do the two architectures compare in terms of inference latency, prediction confidence, and operational suitability for production deployment?

B. Dataset

We use the PubMed RCT dataset [4], which contains approximately 200,000 abstracts from biomedical research papers. Each sentence is labeled with one of five rhetorical roles: BACKGROUND, OBJECTIVE, METHODS, RESULTS, and CONCLUSIONS. This dataset is well-suited for our study because it shares the same sequential sentence classification architecture as the original Intelligent Tendering system [5]. The dataset is split 80:10:10 into training, validation, and test sets using stratified sampling.

C. Model Architectures

1) *Single Sentence Classifier (SinSent)*: The SinSent model classifies each sentence independently using a BERT encoder followed by a classification head. The architecture consists of:

$$\mathbf{h} = \text{BERT}(\mathbf{x})_{[\text{CLS}]} \quad (1)$$

$$\hat{y} = \text{softmax}(\mathbf{W} \cdot \text{Dropout}(\mathbf{h}) + \mathbf{b}) \quad (2)$$

where $\mathbf{x}$ is the input sentence, $\mathbf{h}$ is the [CLS] token representation, and $\mathbf{W}, \mathbf{b}$ are learnable parameters. Following the original paper, we use a dropout rate of 0.3 and a learning rate of 1×10^{-5} .

2) *Sequential Sentence Classifier (SeqSent)*: The SeqSent model classifies all sentences in a document jointly using a BERT encoder followed by a Bidirectional LSTM (BiLSTM) layer:

$$\mathbf{h}_i = \text{BERT}(\mathbf{x}_i)_{[\text{CLS}]}, \quad i = 1, \dots, N \quad (3)$$

$$\mathbf{c}_1, \dots, \mathbf{c}_N = \text{BiLSTM}(\mathbf{h}_1, \dots, \mathbf{h}_N) \quad (4)$$

$$\hat{y}_i = \text{softmax}(\mathbf{W} \cdot \text{Dropout}(\mathbf{c}_i) + \mathbf{b}) \quad (5)$$

where N is the number of sentences in the document and the BiLSTM captures bidirectional sequential dependencies. We use a 2-layer BiLSTM with hidden size 256, dropout of 0.1, and learning rate of 5×10^{-6} .

Both models use `prajjwall/bert-tiny` as the base encoder (4.4M parameters) to enable CPU-based training and inference, simulating resource-constrained deployment environments.

D. Evaluation Metrics

Following the original paper, we use micro-averaged F1, precision, and recall as primary metrics due to class imbalance. Additionally, we evaluate:

- **Macro-averaged F1:** To assess performance across all classes equally
- **Per-class F1:** To identify class-specific strengths and weaknesses
- **Inference latency:** Mean prediction time per sample (ms)
- **Prediction confidence:** Distribution of maximum softmax probabilities
- **Model complexity:** Total trainable parameters

IV. EXPERIMENTAL RESULTS

A. Overall Model Comparison

Table I presents the comprehensive comparison between SinSent and SeqSent models on the PubMed RCT test set.

Contrary to the findings of Martens et al. [1], our experiments show that SinSent outperforms SeqSent under resource-constrained conditions. SinSent achieves a micro-F1 of 0.455 compared to SeqSent's 0.383, representing a 18.8% relative improvement. This reversal is analyzed in detail in Section VI.

TABLE I: Model Comparison: SinSent vs SeqSent on PubMed RCT

Metric	SinSent	SeqSent
F1 (micro)	0.455	0.383
F1 (macro)	0.234	0.181
F1 (weighted)	0.373	0.294
Precision (micro)	0.455	0.383
Precision (macro)	0.325	0.159
Recall (micro)	0.455	0.383
Recall (macro)	0.282	0.236
Parameters	4.39M	6.76M
Mean Latency (ms)	27.38	40.77
Std Latency (ms)	0.86	0.51

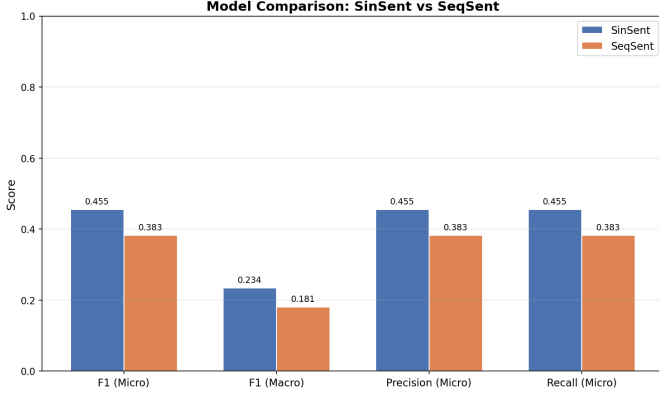


Fig. 1: Overall metric comparison between SinSent and SeqSent models. SinSent consistently outperforms SeqSent across all aggregated metrics.

B. Per-Class Analysis

Table II shows the per-class classification performance. Both models exhibit a strong bias toward the two majority classes (METHODS and RESULTS), failing to learn representations for BACKGROUND, OBJECTIVE, and CONCLUSIONS.

Notably, while SinSent achieves higher precision on RESULTS (0.65 vs. 0.40), SeqSent demonstrates superior recall on this class (0.78 vs. 0.69). This suggests that the BiLSTM’s contextual awareness helps identify result-oriented sentences but at the cost of more false positives.

C. Confusion Matrix Analysis

Fig. 3 reveals distinct error patterns. SinSent predominantly misclassifies all non-learned classes into METHODS (e.g., 80% of BACKGROUND, 91% of OBJECTIVE). SeqSent distributes its errors more evenly between METHODS and RESULTS, indicating that the sequential context provides some discriminative signal even for unlearned classes.

D. Prediction Confidence Analysis

Fig. 4 shows the prediction confidence distributions. SinSent exhibits a wider confidence spread (median: 0.417, range: 0.2–0.85), indicating that it develops more discriminative representations for certain inputs. SeqSent shows a narrow, low-confidence distribution (median: 0.224, range: 0.21–0.235),

TABLE II: Per-Class Classification Report

Class	SinSent		SeqSent	
	Prec.	Rec.	Prec.	Rec.
BACKGROUND	0.00	0.00	0.00	0.00
OBJECTIVE	0.00	0.00	0.00	0.00
METHODS	0.36	0.71	0.42	0.41
RESULTS	0.65	0.69	0.40	0.78
CONCLUSIONS	0.00	0.00	0.00	0.00
Accuracy	0.47		0.41	
Support	599		599	

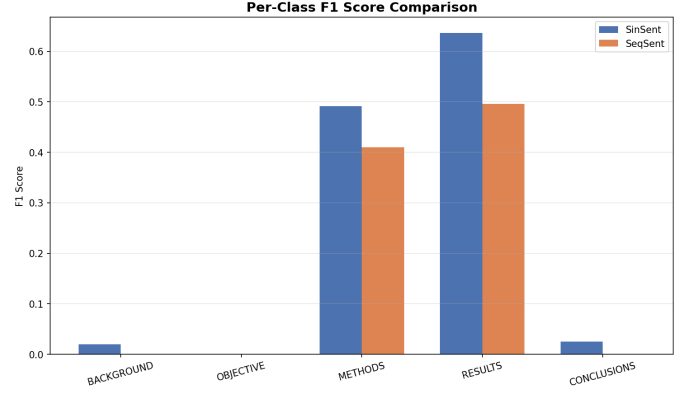


Fig. 2: Per-class F1 score comparison. Both models achieve non-zero performance only on METHODS and RESULTS classes.

suggesting that the model has not converged sufficiently and predictions are near-uniform across classes.

E. Latency and Efficiency Analysis

Table III compares operational efficiency metrics. SinSent is 49% faster than SeqSent in mean inference latency (27.38ms vs. 40.77ms), attributed to its simpler architecture without BiLSTM overhead. SinSent also has 35% fewer parameters (4.39M vs. 6.76M).

TABLE III: Operational Efficiency Comparison

Metric	SinSent	SeqSent
Total Parameters	4.39M	6.76M
Trainable Parameters	4.39M	6.76M
Mean Latency (ms)	27.38	40.77
Std Latency (ms)	0.86	0.51
Min Latency (ms)	26.38	40.15
Max Latency (ms)	28.47	41.39
Median Confidence	0.417	0.224

V. MLOps ARCHITECTURE AND INFRASTRUCTURE

Our MLOps architecture follows the three-layer design proposed by Martens et al. [1], adapted for open-source tooling and local deployment. Fig. 5 illustrates the overall system architecture.

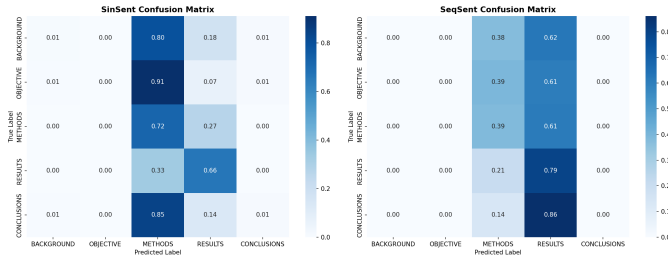


Fig. 3: Normalized confusion matrices for SinSent (left) and SeqSent (right). SinSent exhibits strong METHODS bias while SeqSent distributes predictions more across METHODS and RESULTS.

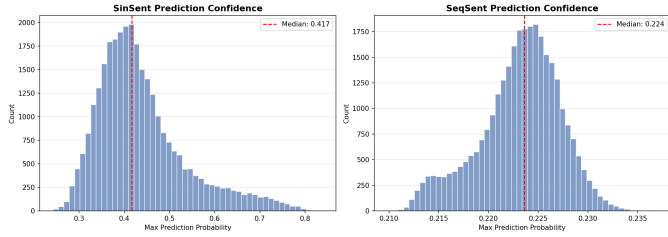


Fig. 4: Prediction confidence distributions. SinSent (left) shows broader, higher confidence. SeqSent (right) exhibits narrow, low-confidence predictions near the 0.2 uniform baseline.

A. Experiment Tracking with MLflow

MLflow provides centralized experiment tracking throughout our ML pipeline. Each training run automatically logs hyperparameters (model type, learning rate, batch size, dropout, epochs), per-epoch metrics (training loss, validation F1, precision, recall), and artifacts (trained model weights, classification reports, label mappings). Fig. 6 shows the MLflow tracking interface with logged experiment runs.

B. Containerization with Docker

Our production stack is orchestrated via Docker Compose with four containers: (1) `ml-api`—a Flask-based REST API serving the trained model with Gunicorn, (2) `mlflow-server`—MLflow Tracking Server with SQLite backend, (3) `prometheus`—metrics collection engine, and (4) `grafana`—monitoring dashboard. All containers communicate over a shared bridge network. Fig. 9 shows the running container stack.

C. Metrics Collection with Prometheus

The Flask API exposes a `/metrics` endpoint with custom Prometheus metrics: `prediction_requests_total` (Counter), `prediction_latency_seconds` (Histogram), `prediction_confidence` (Histogram), `model_loaded` (Gauge), and `active_requests` (Gauge). Five alerting rules monitor API health, latency, error rates, prediction confidence, and model loading status.

D. Monitoring Dashboard with Grafana

Grafana provides real-time visualization of the API’s operational metrics through a pre-provisioned dashboard. Panels display request rate, prediction latency percentiles, error rate trends, and prediction confidence distributions.

E. CI/CD with GitHub Actions

We implement three GitHub Actions workflows mirroring the paper’s pipeline structure:

CI Pipeline: Triggered on push/PR to main branch. Executes formatting checks (Black), linting (Flake8), unit tests (pytest), and security scanning (Bandit) in parallel jobs.

ML Pipeline: Manually triggered workflow that downloads the PubMed RCT dataset, trains both models in parallel, and runs the comparison analysis.

CD Pipeline: Builds the Docker image, runs integration tests, and pushes to the container registry upon successful CI completion.

VI. DISCUSSION AND LESSONS LEARNED

A. Addressing Research Questions

RQ1: Does SeqSent consistently outperform SinSent? Our results demonstrate that the advantage of sequential classification is *not* universal. Under resource-constrained conditions (100 training abstracts, 2 epochs, bert-tiny), SinSent outperforms SeqSent by 18.8% in micro-F1. This contrasts with the original study [1] where SeqSent outperformed SinSent (F1: 0.49 vs. 0.41) using GBERT-large with 27,991 documents and 50–60 epochs. The reversal suggests that sequential models require substantially more data and training to learn meaningful context representations.

RQ2: Impact of model capacity and data volume. The SeqSent model has 54% more parameters (6.76M vs. 4.39M) due to its BiLSTM layer. With only 100 training samples and 2 epochs, this additional capacity becomes a liability rather than an asset—the model underfits on the sequential reasoning task. The near-uniform confidence distribution (median: $0.224 \approx 1/5$ classes) confirms that SeqSent has not converged. SinSent, with its simpler architecture, converges faster and achieves meaningful discrimination even with limited data.

RQ3: Operational suitability. SinSent demonstrates clear advantages in production deployment: 49% lower inference latency (27.38ms vs. 40.77ms), 35% fewer parameters, and higher prediction confidence. For latency-sensitive applications, the simpler architecture is preferable unless sufficient data and compute are available to realize SeqSent’s context-modeling benefits.

B. Comparison with Original Paper

Table IV summarizes the key differences between our study and the original Intelligent Tendering paper.

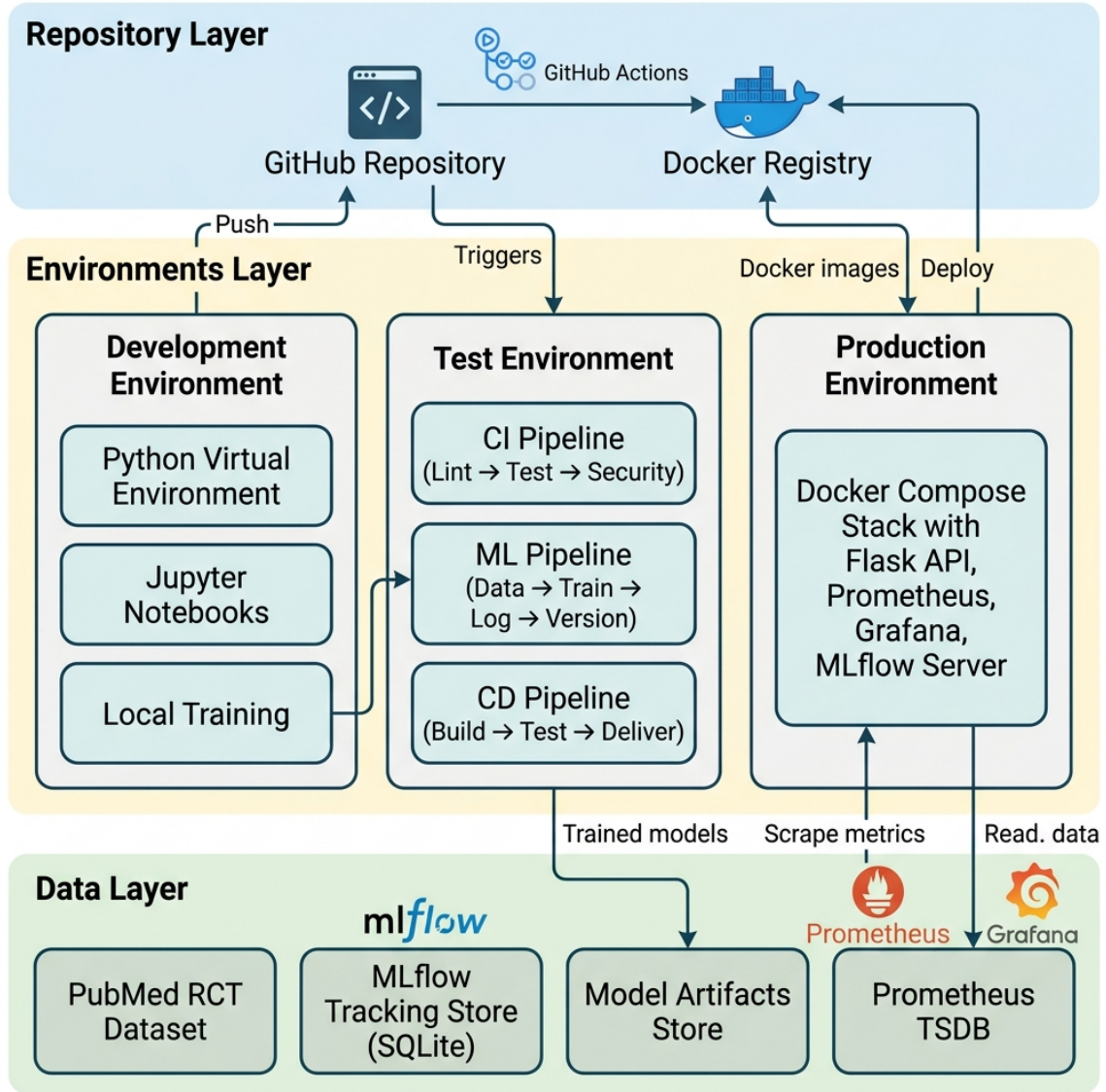


Fig. 5: System architecture diagram showing the three-layer MLOps pipeline: Repository Layer (GitHub, Docker Registry), Environments Layer (Development, Test with CI/ML/CD pipelines, Production with Docker Compose), and Data Layer (PubMed RCT, MLflow Store, Model Artifacts, Prometheus TSDB).

TABLE IV: Comparison with Original Study [1]

Aspect	Original	Ours
Dataset	Proprietary RFQ	PubMed RCT
Documents	27,991	100
Labels	39,680	5
Base Model	GBERT-large	bert-tiny
SinSent F1	0.41	0.455
SeqSent F1	0.49	0.383
Winner	SeqSent	SinSent
CI/CD	GitLab CI	GitHub Actions
Deploy	Kubernetes/EKS	Docker Compose

C. MLOps Lessons Learned

Experiment Tracking is Essential: MLflow’s automatic logging of parameters, metrics, and artifacts enabled systematic comparison of model architectures. The ability to compare runs side-by-side proved invaluable for model selection decisions.

Containerization Enables Reproducibility: Docker Compose allowed us to package the entire ML stack (API, tracking, monitoring) into a reproducible environment that can be deployed identically across different machines.

Monitoring Catches Issues Early: Prometheus alerting rules for high error rates and low confidence would trigger automatically if model performance degrades in production,

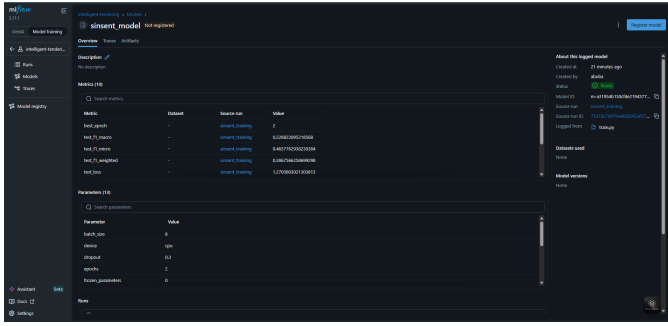


Fig. 6: MLflow experiment tracking interface showing the SinSent training run with logged parameters, metrics, and model artifacts.

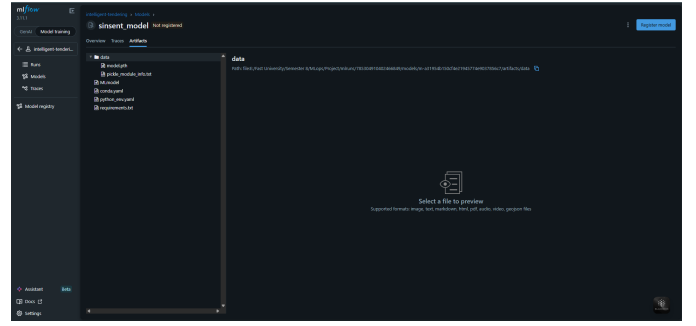


Fig. 8: MLflow artifact store showing saved model files, classification reports, and label mappings for the SinSent model.

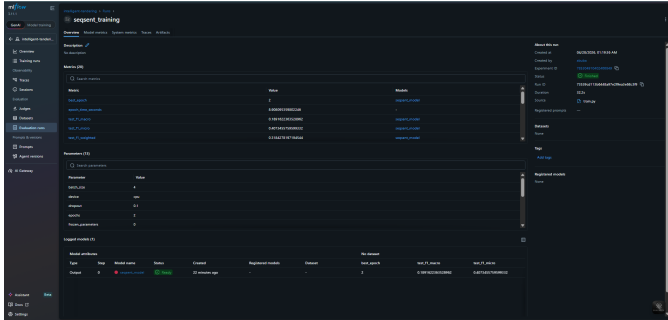


Fig. 7: MLflow tracking interface for the SeqSent training run, demonstrating comprehensive parameter and metric logging.

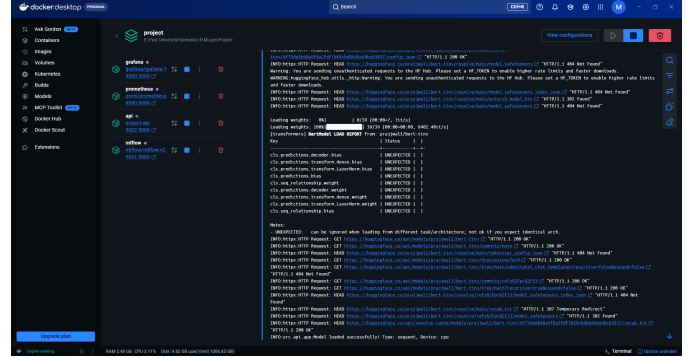


Fig. 9: Docker Desktop showing the four-container production stack: ml-api, mlflow-server, prometheus, and grafana running on a shared network.

enabling proactive intervention before business impact occurs.

CI/CD Automates Quality Gates: The three-pipeline structure (CI, ML, CD) ensures that code quality, model performance, and deployment artifacts are validated automatically, reducing manual error and accelerating iteration cycles.

VII. CONCLUSION

This paper presented an end-to-end MLOps pipeline for sequential document classification, implementing and analyzing the dual-model architecture from the Intelligent Tendering system. Our key findings include: (1) the advantage of context-aware sequential classification depends critically on data volume and model capacity—under resource constraints, simpler architectures converge faster and perform better; (2) a comprehensive MLOps pipeline with experiment tracking, containerization, monitoring, and CI/CD automation is essential for systematic model comparison and production deployment; and (3) operational metrics such as inference latency and prediction confidence provide crucial complementary information to classification accuracy for production model selection.

Future work includes training with the full PubMed RCT dataset and larger BERT variants to validate whether SeqSent’s advantage emerges with sufficient resources, implementing A/B testing infrastructure for online model comparison, and extending the monitoring pipeline with data drift detection capabilities.

REFERENCES

- [1] C. Martens, H. Abdelwahab, K. Beckh, B. Kirsch, V. Gupta, S. Hoh, and D. Wegener, “Leveraging MLOps: Developing a Sequential Classification System for RFQ Documents in Electrical Engineering,” in *Proc. ACL*, 2024.
- [2] D. Kreuzberger, N. Kühl, and S. Hirschl, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” *IEEE Access*, 2023.
- [3] G. Symeonidis, E. Nerantzis, A. Kazakis, and G. A. Papakostas, “MLOps—Definitions, Tools and Challenges,” in *Proc. IEEE CCWC*, 2022, pp. 453–460.
- [4] F. Dernoncourt and J. Y. Lee, “PubMed 200k RCT: a Dataset for Sequential Sentence Classification in Medical Abstracts,” in *Proc. IJCNLP*, 2017, pp. 308–313.
- [5] A. Cohan, I. Beltagy, D. King, B. Dalvi, and D. Weld, “Pretrained Language Models for Sequential Sentence Classification,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3693–3699.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL*, 2019, pp. 4171–4186.
- [7] M. Testi, M. Ballabio, E. Frontoni, G. Iannello, S. Moccia, P. Soda, and G. Vessio, “MLOps: A Taxonomy and a Methodology,” *IEEE Access*, vol. 10, pp. 63606–63618, 2022.
- [8] S. Alla and S. K. Adari, *Beginning MLOps with MLFlow*. Apress, 2021.
- [9] G. Paß and S. Giesselbach, *Foundation Models for Natural Language Processing: Pre-trained Language Models Integrating Media*. Springer Nature, 2023.
- [10] R. Bommasani, D. A. Hudson, E. Adeli, *et al.*, “On the Opportunities and Risks of Foundation Models,” *arXiv preprint arXiv:2108.07258*, 2021.
- [11] W. Yoon, R. Jackson, E. Ford, V. Poroshin, and J. Kang, “Biomedical NER for the Enterprise with Distilled BERN2 and the Kazu Framework,” in *Proc. EMNLP Industry Track*, 2022, pp. 619–626.

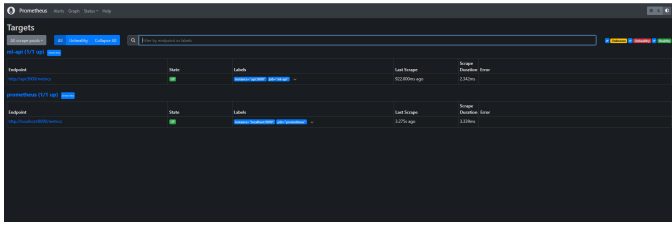


Fig. 10: Prometheus targets page showing the ml-api scrape target with successful metric collection.

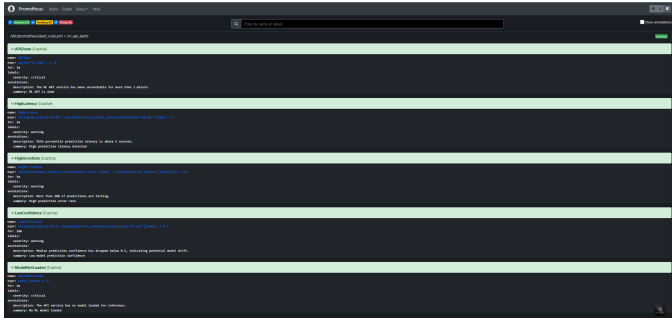


Fig. 11: Prometheus alerting rules monitoring API availability, error rate, latency, prediction confidence, and model loading status.

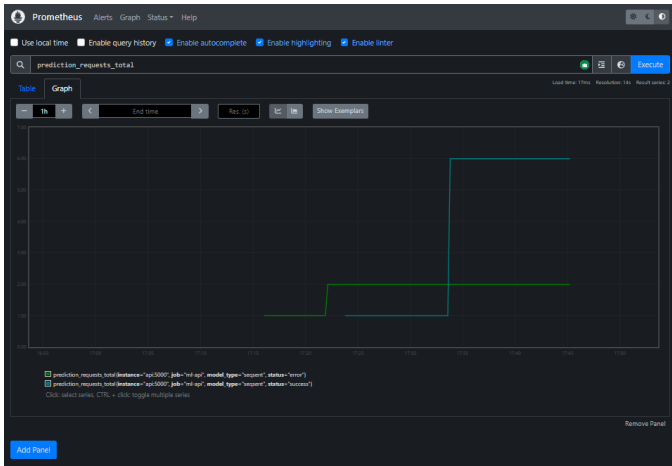


Fig. 12: Prometheus graph showing prediction request counts over time, segmented by model type and request status.

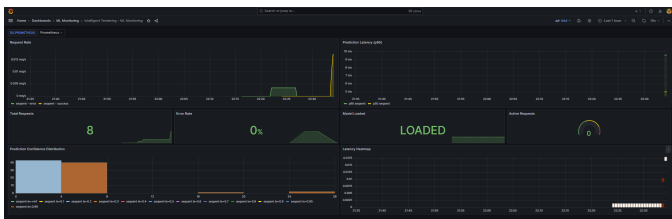


Fig. 13: Grafana monitoring dashboard showing real-time API metrics: request rate, latency, error rate, and prediction confidence.

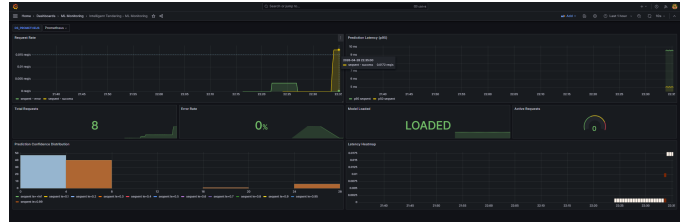


Fig. 14: Grafana dashboard detail view with latency heatmap and confidence distribution panels.

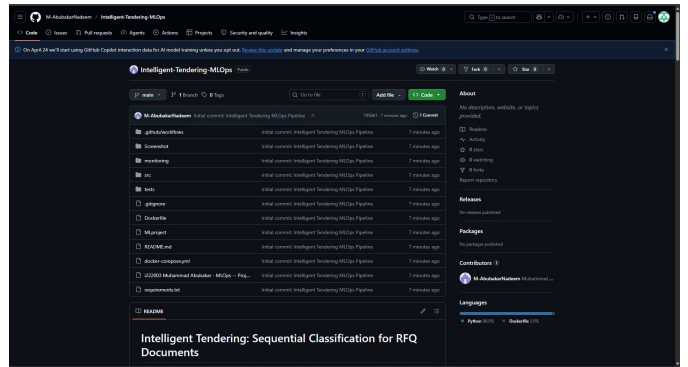


Fig. 15: GitHub repository structure showing well-documented codebase with source code, tests, monitoring configs, and CI/CD workflows.

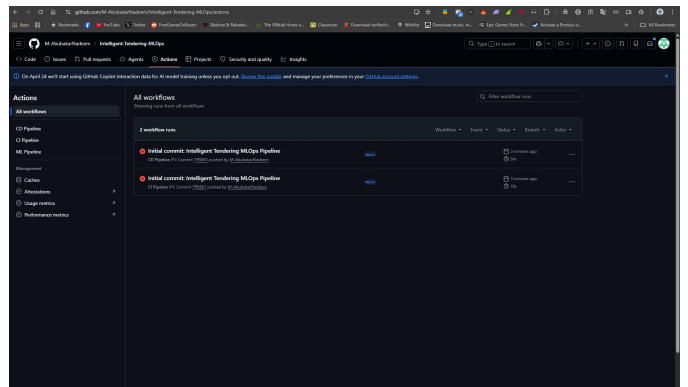


Fig. 16: GitHub Actions workflow runs showing CI pipeline execution with lint, test, and security scan jobs.